manvendra@rajpoot.dev

Manvendra Rajpoot

github.com/alzywelzy

+91 94522 77560

rajpoot.dev

linkedin.com/in/alzywelzy

Dear Hiring Manager,

I am writing to apply for the Frontend Developer role on your team. I build interfaces for products where the frontend carries real weight: billing consoles, cloud control panels, offline-capable apps and fast, accessible marketing sites. Because I also build the APIs behind them, I design screens around how the data actually behaves, not around a mock.

At CloudTechTiq I built the customer and admin console for a cloud hosting and billing platform in React 18, Redux Toolkit and Vite, across more than 35 feature modules: billing and invoices, orders and carts, CloudStack resources, tickets, staff roles, tax and payment settings, and monitoring dashboards in ApexCharts. I shipped an in-browser virtual-machine console on xterm.js, Razorpay checkout, Microsoft sign-in through MSAL, Cloudflare Turnstile, admin impersonation and sanitised rich-text content, all behind axios interceptors that handle retries and silent JWT refresh. I also built the company's marketing site in Next.js and TypeScript with Tailwind and Radix/shadcn components, and the Workforce HR web app in React 19, TypeScript and Tailwind v4 with i18n, a documented design system, and Vitest and Playwright tests.

I have built offline-first patterns in React 19 and TypeScript: TanStack Query over an IndexedDB outbox that replays writes when the connection returns, installable as a progressive web app, and covered by Vitest unit tests and Playwright end-to-end suites against the real API. I keep frontend and backend in lockstep with typed TypeScript API layers, schema-validated contracts in Zod and Pydantic, and OpenAPI specs, so a contract change fails in CI rather than in a customer's browser. For public pages I ship fast static surfaces in Next.js and Astro with generated Open Graph images, RSS and sitemaps.

My portfolio, rajpoot.dev, shows how I treat quality on the web. It is Next.js 16, React 19 and strict TypeScript, statically prerendered, with a strict Content Security Policy, JSON-LD and Lighthouse CI budgets on script and transfer size. CI runs ESLint, Vitest with a coverage gate, Playwright across Chromium, WebKit and mobile, CodeQL and visual regression. I care about accessibility, reduced motion, and keeping the JavaScript a page ships as small as it can be.

Because I own the backend too (Django REST Framework, FastAPI, payments and authentication), I can carry a feature from schema to pixel and spot contract problems before they reach the user. I am comfortable reading a Django serializer or a FastAPI schema, proposing the change the interface needs, and shipping both sides together, which keeps the frontend honest about loading, error and empty states instead of designing only for the happy path.

I would welcome the chance to discuss how this experience fits your team, and I am happy to walk through any of the systems above in technical detail. Thank you for your time and consideration.

Sincerely,

Manvendra Rajpoot